Aula #1 (Introdução da linguagem Java)

- Precursores da linguagem Java
    - Java é uma das linguagens que evoluiu a partir da linguagem C
    - C++ é o C estendido, ou C com orientação a objetos.
    - Java aproveitou boom de páginas Web interativas c/ cont. dinâmico.
    - Utilizada para projetos grandes, os quais linguagens como C podem não ser tão adequadas.
    - Alguns programas exigem que o dispositivo tenha o Java instalado
- A linguagem Java
    - Os programas Java são classes, que possuem atributos e métodos
    - Podemos criar nossas classes bem como utilizar a API de Java
- Vantagens e desvantagens de Java
    - Link sobre vantagens e desvantagens
        - simplicidade, comparado com outras linguagens (e.g. C++)
        - bom suporte ao paradigma orientado a objetos
        - segurança (encapsulamento, ausência de ponteiros, GC auto.)
        - portabilidade, redução de custos de desenv. e manutenção
        - multithreading
        - performance abaixo de outras linguagens
        - pode soar um pouco verborrágica
        - custosa no quesito consumo de memória
- Escrevendo um programa Java
    - edição: escrito em editor, armazenado em disco rígido (.java)
    - compilação: javac, converte código em bytecodes, executado pela JVM. Bytecodes independentes de plataforma. Bytecodes em arquivos .class.
    - carregamento: carregador de classe da JVM move os bytecodes para a memória primária
    - verificação: verificador de bytecodes confirma que todos os bytecodes são válidos e não violam restrições de segurança de Java
    - execução: JVM lê os bytecodes, compilando o que foi especificado. Essa compilação ocorre de modo *just-in-time* (JIT)
    - OBS: Java combina interpretação com a compilação *just-in-time*
        - procura-se partes que executam com frequência e compila essas partes, executando o código compilado quando for necessário no futuro
- JVM (Java Virtual Machine)
    - máquina virtual, simula um hardware. Dispositivos com a mesma JVM podem executar código Java, conferindo alta portabilidade.
- Link sobre a linguagem Java.

Aula #2 (Primeiro programa Java e conceitos básicos sobre a linguagem)

- Definição de uma classe
    - public class NomeDaClasse
    - Os programas Java são escritos dentro de classes
    - Todo programa Java contém pelo menos uma classe
- Criação do método main(String[] args)
    - No main, colocamos o código que desejamos “rodar” ao executá-lo
- Impressão na saída padrão com System.out.println();
    - System.out nos ajuda a operar com a saída padrão da linguagem
- Comentários de uma linha, de mais de uma linha e comentários Javadoc (/**)
    - Mais adiante, veremos como gerar documentação Java com javadoc
- Erros de sintaxe e erros de semântica
    - sintaxe: mais relacionado com estruturas do código
    - semântico: mais relacionado com significados dessas estruturas
- Regras e boas práticas de nomenclatura de classes e variáveis
    - variáveis: nomes significativos, começa com minúsculo e outras em maiúsculo
    - classes: nomes significativos, começa com maiúsculo cada palavra e as demais letras em minúsculo
    - em Java, a classe e o nome do arquivo devem ser os mesmos
- Instruções em Java (;)
    - diferente de Python, semelhante a C
- Modificações nas impressões: o método print e caractere de escape ‘\n’
    - println imprime a mensagem e, logo depois, quebra a linha
    - print imprime a mensagem mas mantém o cursor no final
- O método printf (%d, %s, etc)
    - permite uma saída formatada, semelhante ao printf de C
- A classe Scanner de java.util.Scanner
    - java.util.Scanner precisa ser localizado (import)
- Criação de objetos Scanner(System.in);
- A palavra-chave new
- Definição de variáveis em Java
    - precisam de tipo e nome, semelhante à linguagem C
- Captura de entradas com next() (e.g. nextInt, nextLine)
- Conceitos de memória do computador
    - locais de memória nomeados (com valores atribuídos)
- Operações aritméticas e precedência de operadores
- Conversão de tipos (explícita e implícita) e operadores de coerção (e.g. (int))
- Operadores de igualdade e operadores relacionais (==, !=, >, etc.)
- O tipo boolean
- Tipagem de variáveis
- Linguagem fortemente tipada
- Link sobre linguagens estaticamente e dinamicamente tipadas.
- Exemplo: Código #01

Aula #3 (Condições e loops em Java)

- if, if.. else, if.. else if.. else
- Operador ternário (“?” e “:”)
- while
    - repetições controladas por contador
    - repetições controladas por sentinela
- Operadores de incremento e decremento (++ e –): a++, ++a
- Operadores de atribuição compostos (+=)
- do.. while
- for
- switch (switch(c){case 1: … break; case 2: … break; default: …}
- break e continue
- Operadores lógicos (&&, ||, !)
- Exemplo: Código #02

Aulas #4 (Noções básicas de orientação a objetos)

- Especificação de entidades (do mundo real ou não)
- As entidades possuem atributos e comportamentos (campos e métodos)
- A diferença entre classe e objeto
    - classes são especificações de alguma coisa (atributos e métodos)
    - objetos são instâncias dessas especificações
- Definição e utilização de métodos construtores
    - Os construtores não possuem retorno
    - O construtor padrão (inicializa variáveis c/ valores padrão)
    - Podemos definir novos construtores (o padrão deixa de existir)
- A chamada de método envia mensagens para um objeto
    - obj.mensagem() é um envio de mensagem para o objeto
- Criação de classes básicas e instanciação básica de objetos
- Definição de atributos e métodos básicos
- Interação com objetos através dos métodos
- Criação de métodos com parâmetros
- Tipos primitivos e tipos por referência (null)
    - as variáveis de tipos por referência referenciam objetos na memória
    - tipos primitivos (Java): boolean, byte, char, short, int, long, float, double
    - observe que string não faz parte dos tipos primitivos em si
- Tipos primitivos inicializados por padrão quando variáveis de instância
- Variáveis de instância c/ tipos de referência iniciados p/ padrão como null.

Aulas #5 e #6 (Controle de acesso a atributos e métodos)

- Variáveis de instância e métodos get e set
    - cada instância de uma classe possui seu valor p/ var. de instância
    - métodos get nos ajudam a obter esses valores em certos contextos
    - métodos set nos ajudam a mudar esses valores em certos contextos
- Modificadores de acesso public e private
    - public: permite que todas as classes acessam
    - private: permite que apenas a própria classe acesse (ocult. de info.)
- Abstração e encapsulamento
    - abstração envolve se concentrar no que é relevante no contexto
    - encapsulamento envolve esconder detalhes de implementação
- Os métodos que utilizamos sem dar conta de como foram implementados
- Reutilização de código via métodos
- Exemplos de classes com métodos get e set
- Escopo de variáveis
- Referenciando membros do próprio objeto com this (referência a si mesmo)
- Link sobre encapsulamento de código.
- Exemplo do Código #03

Aula #7 (Atributos e métodos estáticos)

- Conceitos gerais sobre atributos e métodos static
- Atributos estáticos
    - variáveis de classe, com valor único para todas as variáveis
    - diferem de variáveis de instância, onde cada instância possui um valor
- A existência desses membros enquanto a classe está carregada na memória
- Métodos estáticos
    - métodos acessíveis a partir da própria classe
    - não são invocados a partir de um objeto, por assim dizer
- Quando utilizar atributos e métodos estáticos? Quando não utilizar?
    - atributos estáticos podem ser úteis quando se deseja ter um valor comum para todas as instâncias, atualizado para todas sem a necessidade de acessar e alterar uma a uma
    - atributos estáticos possuem comportamento semelhante a uma variável global, o que pode ocasionar em problemas
    - métodos estáticos podem ser úteis quando se deseja utilizar uma implementação que independe de instância
- Exemplos de atributos e métodos estáticos
- Pilha de chamadas de métodos
- Link sobre atributos estáticos em Java. Outro link sobre o mesmo assunto. Por fim, um link sobre quando utilizar métodos estáticos.

Aula #8 (Sobrecarga de métodos)

- Sobrecarga (overload) de métodos e sobrecarga de construtores

    - parâmetros distintos (em quantidades, em tipos, em ordem)
- Os métodos não são distinguíveis por tipo de retorno
    - exemplo de métodos com mesmos parâmetros mudando retorno
- Assinatura de métodos e determinação dos métodos corretos
- Exemplos de métodos construtores
    - diferentes construtores funcionando de forma diferente
- Exemplos de sobrecarga de métodos
- Link sobre sobrecarga de métodos em Java.
- Exemplo do Código #04

Segundo nosso planejamento, teremos revisão de conteúdo e, então, a prova 1-1.

Aula #9 (Arrays na linguagem Java)

- Arrays em Java
- Acesso indexado e alteração através de índice
- A variável de instância length
    - obtenção do tamanho de um array em Java
- Declaração de arrays em Java (new int[10], nada, {3, 2})
- Aplicações de arrays e estruturas de controle (condicionais e loops)
    - e.g. percorrer todos os elementos de um array e fazer alguma coisa
- Arrays passados para métodos
- Passagem por valor e passagem por referência
    - em Java, todos os argumentos são passados por valor
    - podem ser cópias de valores (primitivos) ou cópias das referências (tipos de referência), ao invés dos objetos em si
    - interação com métodos públicos deste objeto podem alterá-lo
- Alteração de valores através dos métodos public dentro de métodos
- ArrayList: o que são, suas vantagens e diferenças com relação aos arrays
    - tamanho dinâmico, métodos para operar com ArrayLists
    - alguns métodos de ArrayList: add, remove, contains, get, indexOf
    - ArraysList genérico (<E>)
- Documentação: https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html
- foreach (for tipo elemento : lista)
- Link com documentação de ArrayList.
- Link sobre passagem por valor e por referência em Java.
- Link sobre classes empacotadoras (wrappers) em Java.
- Exemplo em Código #05

Aula #10 (Herança: parte I)

- Aproveitamento de implementação entre classes
    - classes podem aproveitar implementações de outras classes

    - classes podem também receber novas implementações (estender)
- Superclasses e subclasses
- Superclasses diretas e indiretas
- Object como superclasse de todas as classes Java
    - e.g. toString() é utilizável por qualquer classe Java
- O conceito de estender (extends) e classes mais ou menos específicas
- O relacionamento “é um”
- Modificador de acesso protected (superclasse, subclasses e mesmo pacote)
- Modificador private e herança
- Classes abstratas e métodos abstratos (public abstract int/class)
- Quando utilizar classes e métodos abstratos?
    - repassar responsabilidade de implementação para subclasses
    - proibir a instanciação de certas classes
    - em aulas futuras, faremos uma comparação com interfaces
- Exemplos envolvendo classes e métodos abstratos
- Utilização de tipos abstratos na declaração de variáveis
- Exemplo em Código #06

Aula #11 (Herança: parte II)

- Sobrescrita de métodos (override) e notação @Override antes do método
    - substituir implementação de superclasses nas subclasses
- Construtores em subclasses
- Chamando métodos de superclasses em subclasses com super
    - e.g. construtores da superclasse super(...)
- Inicialização de campos da superclasse com o construtor da superclasse

Aula #12 (Polimorfismo através de herança)

- O que é polimorfismo?
    - programar no geral, para objetos que compartilham uma superclasse
    - considerar objetos de classes “como se fossem” da superclasse
- Emissão da mesma mensagem mas obtenção de resultados distintos
    - o resultado irá depender do tipo do objeto
- Upcasting e downcasting
    - upcast é conversão para cima (subclasse para superclasse)
        - procedimento seguro, pela relação “é um”
        - por vezes, acontece de forma implícita (sem declaração)
    - downcast é conversão para baixo (superclasse para subclasse)
        - permite acesso a métodos da subclasse fora da superclasse
        - nem sempre é seguro, ocorre sempre de forma explícita
- Conversão explícita e conversão implícita
- Operador instanceof (obj instanceof Type)
- Variáveis (atributos) final

    - possuem valores constantes
- Métodos final (implicitamente, private e static são final também)
    - não podem ser sobrescritos (*overridden*)
- Classes final (e.g. String)
    - não podem ser herdadas (e.g. String)

Segundo nosso planejamento, teremos revisão, prova 1-2 e, depois, reposição.

Aulas #13 e #14 (Interfaces)

- O que é uma interface?
    - conjunto de métodos que podem ser chamados em um objeto, mas que não fornece implementações para esses métodos
    - pode ser aplicado para coisas bastante diferentes mas que compartilham serviços em comum
    - pode ser utilizado para conseguir polimorfismo para classes não relacionadas, o que garante para as interfaces uma maior flexibilidade
- Criação de interfaces
    - public interface, definição de métodos, sem necessidade de public
- Polimorfismo através de interfaces
- Exemplos de programas com interfaces
- Os pontos positivos do uso de interfaces
    - pode-se implementar mais de uma interface
    - polimorfismo envolvendo classes que são relacionadas apenas por proverem o mesmo serviço
- Entendendo como classes Java utilizam interfaces (e.g. Comparable)
- Exemplo em Código #07

Aula #15 (Herança, interface, composição e polimorfismo)

- Comparando interfaces com herança
    - uma classe Java só herda diretamente de uma classe Java
    - uma classe Java pode implementar qualquer nº de interfaces
    - herança é uma forma de reaproveitar código (e.g. campos, métodos)
    - parte da implementação pode ficar “em aberto” (abstract)
    - interfaces definem um “contrato de serviço”
    - herança gera um forte acoplamento entre superclasse e subclasse
- Composição: uma classe “tem um” algo que remete a outra classe
    - e.g. Pessoa tem um Papel, Papel pode mudar em runtime
- Interfaces ou classes abstratas?
- Programar para uma interface, não para uma implementação
- Quando utilizar herança e quando utilizar interface?
    - herança é favorecida pelo aspecto do reaproveitamento de código

    - herança é favorecida quando há hierarquia de classes e algo não muda o que é
    - quando não se observa esse tipo de benefício, interfaces tem um uso mais favorável, já que gera menos acoplamento
- Exemplo em Código #08

Aulas #16 e #17 (Coleções em Java)

- Relembrando ArrayList
    - tamanho dinâmico, métodos que operam nesses objetos
- Outros tipos de coleção: Set, List, Map (dicionário)
    - Set: elementos não repetidos, ordem não garantida, foreach
    - Map: par chave-valor, cada um tem um tipo, foreach nas keys
- Boxing (tipo primitivo para empacotador, int -> Integer) ou unboxing
- Ocorrência de autoboxing ao inserir elementos aos Lists
- Link com a documentação de Set, HashSet, Map e HashMap.
- Exemplo em Código #09

Aula #18 (Tratamento de exceções: parte I)

- A exceção como algo incomum de acontecer
- Exemplo da divisão por zero e do NullPointerException
- Criação de bloco try
- Capturando exceções com catch
    - capturando com Exception
    - capturando Exceptions específicas
    - capturando Exceptions envolvendo hierarquia de Exceptions
- O bloco finally
- Fluxo de execução e blocos try/catch/finally
- A cláusula throws em métodos
    - checked e unchecked Exceptions
- throw (forma de retornar exceções)
- Quando utilizar tratamento de exceções?

Aula #19 (Tratamento de exceções: parte II)

- Hierarquia de exceções no Java
- Exceções verificadas e não verificadas (checked e unchecked)
- Criação de nossas próprias exceções (extends Exception)
- Os métodos getMessage e getStackTrace

Segundo nosso planejamento, teremos aula de revisão e, depois, a prova 2-1. A nossa prova escrita não contempla o assunto de tratamento de exceções.

Aulas #19 e #20 (UML - *Unified Modeling Language*)

- O que é UML?
    - ajudar na modelagem e documentação de sistemas
    - e.g. que entidades existem, como procedimentos funcionam
- Diagramas diferentes para diversas representações
    - Diagramas estruturais
        - de classes (classes, atributos, métodos, composição, herança)
    - Diagramas comportamentais
        - de caso de uso (atores, casos de uso)
        - de sequência (ordem das ações/mensagens)
        - de estados (estados, ações, transições)

Aula #21 (Testes de unidade)

- Teste de unidade: o que são, para que servem, vantagens
- Automatização de testes, refatoramento, framework de testes
- *Test-driven development* (TDD)
- Introdução ao framework JUnit
- import junit.framework.* ClasseDeTeste extends TestCase
- Criação de testes para os métodos da classe
- protected void setUp() para inicialização dos campos da classe de teste
- Métodos assert (assertEquals, assertTrue)
- Testar caminhos, valores limite, etc
- Link para documentação de JUnit 4 (podem utilizar outras versões)
- Exemplo em Código #10

Códigos de exemplo (copiem para seus scripts)

## Código #01 - utilização de Scanner

```
// Importação de Scanner (que utilizaremos para ler da entrada padrão)
import java.util.Scanner;

// O nome de classe começa com letra maiúscula
// Caso tenha mais de uma palavra, começar cada uma com maiúscula
// Palavras juntas (sem underline)
// O nome da classe igual ao nome do .java

public class ClasseDeExemplo {

	// Quando formos rodar o programa (Run), executaremos o main

	public static void main(String[] args)
	{
		// Descomente aqui o exemplo que você deseja executar

		//ExemploScanner1();
		//ExemploScanner2();
		//ExemploMedia1();
		//ExemploMedia2();
		//ExemploComparadores();

	}

	public static void ExemploScanner1()
	{
		// Criando um objeto do tipo Scanner
		Scanner sc = new Scanner(System.in);

		// Exibindo mensagens na saída padrão
		System.out.println("Digite um nome: ");

		// Capturando a próxima palavra lida
		String nome1 = sc.next();

		// printf funciona de forma bastante similar ao printf de C
		System.out.printf("O nome informado é: %s", nome1);
	}

	public static void ExemploScanner2()
	{

		// CRIAÇÃO DE OBJETOS (new)
		// Veremos melhor mais adiante como o new funciona

		Scanner sc = new Scanner(System.in);
```

```
            System.out.println("Digite o nome completo: ");

            // NOMES DE VARIÁVEIS EM JAVA
            // Primeiro nome minúsculo, os demais começando com
maiúsculo

            // LEITURA DE LINHA (nextLine)

            String nomeCompleto = sc.nextLine();
            System.out.println("O nome completo é: " + nomeCompleto);

            // LEITURA DE STRINGS SEPARADAS POR ESPAÇO (next)
            // \n é quebra de linha

            System.out.println("\nDigite um nome e sobrenome: (sep. por
espaço)");
            String nome = sc.next();
            String sobrenome = sc.next();
            System.out.printf("O nome informado é: %s\n", nome);
            System.out.printf("O sobrenome informado é: %s\n",
sobrenome);

            // LEITURA DE INTEIROS (nextInt)

            System.out.printf("\nDigite a idade de %s %s\n", nome,
sobrenome);
            int idade = sc.nextInt();
            System.out.printf("A idade de %s %s é %d", nome, sobrenome,
idade);
      }

      public static void ExemploComentarios()
      {
            //Comentário de uma linha
            //Imprime "olar" e quebra a linha
            //System.out.println("olar");

            /*Comentário de mais de uma linha
              Bloco de comentários
            */

            /**Comentário Javadoc (veremos mais adiante)
             *
             */
      }

      public static void ExemploMedia1()
      {
            int nota1 = 10;
            int nota2 = 7;
            System.out.println("A média é:");
```

```
            // Conversão de tipos (testem sem o (float)
            // Erros sintáticos e semânticos (significado)
            // Sem o float, média é calculada errado
            // O erro é semântico (sendo sintático, programa nem
compilaria)

            // (float) antes de nota1 a converte em float

            System.out.println(( (float) nota1 + nota2) / 2);

            // A instrução abaixo gera um erro (descomente para ver)
            //System.out.printf("A média é %.2f", (nota1+nota2)/2);


      }

      public static void ExemploMedia2()
      {
            // Exemplo de conversão implícita (sem o (float))
            float nota1 = 10; //"10" é int, mas é convertido em float
            int nota2 = 7;
            System.out.println(( nota1 + nota2) / 2);
      }

      public static void ExemploComparadores()
      {
            System.out.println(20 == 20);
            System.out.println(20 % 2 > 1);
            System.out.println(1 != 21 % 2);
      }

}
```

## Código #02 - Exemplos com classes básicas

```
// Definindo a especificação de Aluno (atributos e métodos)

public class Aluno {

      // Definindo os atributos da classe Aluno
      public String nome;
      public String matricula;
      public float cra;
      public String email;

      // Definindo os métodos da classe Aluno

      public void exibeInfo()
      {
```

```
            System.out.printf("Nome: %s\nMatrícula: %s\nCRA:
%.2f\nE-mail: %s\n", nome, matricula, cra, email);
      }

      public void setEmail(String novoEmail)
      {
            email = novoEmail;
      }

}

public class CartaDeBaralho {

      // Atributos de uma carta de baralho
      // Toda carta de baralho terá esses atributos
      // Porém, as cartas podem ter valores diferentes para esses
atributos

      public String valor;
      public String naipe;

}

import java.util.Scanner;

public class ClasseDeExemplo {

      public static void main(String[] args)
      {

            // Implementação básica de uma batalha de turnos alternados

            boolean turnoP1 = true;

            Scanner sc = new Scanner(System.in);

            Personagem p1 = new Personagem("Bill", 100, 10);
            Personagem p2 = new Personagem("Gill", 90, 14);

            // Enquanto os dois personagens tiverem com alguma saúde
            while(p1.pv > 0 && p2.pv > 0)
            {
                  if(turnoP1)
                  {
                        p1.atacar(p2);
                  }
                  else
                  {
                        p2.atacar(p1);
                  }
```

```
                p1.exibeInfo();
                p2.exibeInfo();
                System.out.println();
                turnoP1 = !turnoP1;

                // Apenas para parar a execução (digite Enter para
continuar)

                sc.nextLine();
            }


            /**
            Aluno a1 = new Aluno();
            a1.nome = "Denis";
            a1.matricula = "212080296";
            a1.email = "denis0079@hotmail.com";
            a1.cra = (float) 9.10;  //ou 9.10f

            a1.exibeInfo();
            a1.setEmail("denis_william@aluno.uepb.edu.br");
            a1.exibeInfo();

            System.out.println();
            Aluno a2 = new Aluno();
            a2.nome = "Toin";
            a2.matricula = "231080187";
            a2.cra = 0f;

            a2.exibeInfo();
            Aluno a3 = null;
            System.out.println(a3.nome);
            */


            /**
            CartaDeBaralho carta1 = new CartaDeBaralho();
            carta1.valor = "7";
            carta1.naipe = "Paus";

            System.out.printf("A carta1 é %s de %s\n", carta1.valor,
carta1.naipe);

            CartaDeBaralho carta2 = new CartaDeBaralho();
            carta2.valor = "9";
            carta2.naipe = "Copas";

            System.out.printf("A carta2 é %s de %s\n", carta2.valor,
carta2.naipe);
            */
```

```
            /**
            int numero = 0;
            while(numero < 5)
            {
                  System.out.println(numero);
                  numero++; // numero = numero + 1;
            }

            for(int i = 0; i < 5; i++)
            {
                  System.out.println(numero);
            }
            */


            /**
            String senha;
            Scanner sc = new Scanner(System.in);
            System.out.println("Digite uma senha válida (6+
caracteres)");
            while(true)
            {
                  senha = sc.nextLine();
                  if(senha.length() >= 6)
                  {
                        break;
                  }

                  System.out.println("Senha inválida");

            }

            /*
            System.out.println("Digite um número: ");
            Scanner sc = new Scanner(System.in);
            float nota = sc.nextFloat();
            if(nota < 4)
                  System.out.println("O aluno foi reprovado direto");
            else if(nota >= 4 && nota < 7)
                  System.out.println("O aluno vai pra final");
            else if(nota >= 7 && nota <= 10)
                  System.out.println("O aluno foi aprovado");
            else
                  System.out.println("Nota inválida");
             */
      }
}

import java.util.Random;
```

```
// Definindo a especificação de um Personagem em um jogo

public class Personagem {

      // Definindo os atributos do Personagem

      public String nome;
      public int pv;
      public int forca;

      // Definindo um construtor para a classe Personagem (veremos mais
sobre)

      public Personagem(String n, int vida, int ataque)
      {
            nome = n;
            pv = vida;
            forca = ataque;
      }

      // Definindo uma ação para o Personagem

      public void atacar(Personagem alvo)
      {
            Random rand = new Random();
            alvo.pv -= forca + rand.nextInt(-5, 5);
      }

      public void exibeInfo()
      {
            System.out.printf("Nome: %s; PV: %d\n", nome, pv);
      }

}
```

## Código #03 - Aluno com conceitos de controle de acesso implementados

```
// Definindo a especificação de Aluno (atributos e métodos)

public class Aluno {

      // Definindo os atributos da classe Aluno
      // Nós não queremos que outras classes acessem ou manipulem esses
      // valores diretamente. Por conta disso, faremos com que apenas a
      // própria classe Aluno acesse e manipule diretamente esses
atributos.
      // Fazemos isso definindo-as como privadas (private)

      private String nome;
```

```
      private String matricula;
      private float cra;
      private String email;

      // Construtor da classe Aluno
      // Podemos definir os valores de nome, matricula, cra e email no
ato
      // da criação do objeto.
      // O método construtor tem o mesmo nome da classe e cria um
objeto da
      // classe Aluno.
      // Caso nenhum construtor tivesse sido definido, poderíamos
utilizar
      // um construtor padrão, sem parâmetros, que não define nenhum
valor
      // e apenas cria esse objeto dessa classe.


      public Aluno(String n, String m, float c, String e)
      {
            // o "this" diz respeito ao próprio objeto. Nesse caso,
this.nome
            // diz respeito ao nome do objeto que está sendo criado.

            this.nome = n;
            this.matricula = m;
            this.cra = c;
            this.email = e;
      }


      // Definindo os métodos da classe Aluno. Para chamar esse método,
      // precisamos de um objeto da classe Aluno. Sendo, por exemplo,
esse
      // objeto um aluno "a", faremos a.exibeInfo() para exibir as
infos de a

      public void exibeInfo()
      {
            System.out.printf("Nome: %s\nMatrícula: %s\nCRA:
%.2f\nE-mail: %s\n", this.nome, this.matricula, this.cra, this.email);
      }

      // Método público que me permite acessar o valor do nome do
objeto
      // Aluno em outra classe. Esse acesso é indireto, ou seja, outra
      // classe não consegue acessar o nome do aluno diretamente
(a.nome).
      // Ela só conseguirá fazer isso "consultando" o aluno, que cede
essa
```

```
	// informação através do getNome. É uma forma de conseguir
encapsulamento,
	// já que o atributo nome está encapsulado, protegido, não pode
ser
	// acessado por outras classes.

	public String getNome()
	{
		return this.nome;
	}

	// Método que permite modificarmos o nome do aluno. Novamente,
essa
	// modificação é indireta. Outra classe só consegue mudar o nome
do
	// aluno através desse método (ou outros métodos públicos que
permiti-
	// ssem essa mudança). Nesse caso, o valor é mudado independente
de
	// qual nome passarmos, mas poderíamos fazer algum tratamento.

	public void setNome(String n)
	{
		this.nome = n;

	}

	public float getCRA()
	{
		return cra;
	}

	// O método setCRA chama um método auxiliar que faz o tratamento
de
	// CRAs inválidos. Lembre que getCRA faz uma determinada função,
mas
	// outras classes não precisam ter detalhes disso
(encapsulamento,
	// ocultação de informação). Assim como utilizamos vários métodos
Java
	// que sabemos o que fazem mas não como fazem (e muitas vezes não
nos
	// importamos com isso).

	public void setCRA(float c)
	{
		cra = trataCRA(c);
	}

	// O método trataCRA é um método auxiliar que faz um
subprocedimento
```

```
	// dentro de aluno, e esse subprocedimento é irrelevante para
outras
	// classes (ou podemos forçar que não possam ser utilizados em
outras
	// classes). Conseguimos isso utilizando o modificador de acesso
	// private para o método.

	private float trataCRA(float c)
	{
		if(c >= 0 && c <= 10)
			return c;
		else if(c < 0)
			return 0;
		else
			return 10;
	}

	// Esse exemplo mostra a utilização do this. Podemos utilizar
this
	// para deixar de forma bem clara que o primeiro valor diz
respeito
	// ao objeto a partir do qual estamos invocando o método.

	public float mediaCRAOutroAluno(Aluno outro)
	{
		return (this.cra + outro.cra)/2;
	}

	// Lembrar de fazer get e set para os atributos da classe. Em
algumas
	// situações específicas, pode ser que vocês acabem criando
apenas
	// o get, apenas o set, ou até mesmo nenhum

}

public class ClasseDeExemplo {

	public static void main(String[] args)
	{

		/* Perceba que podemos criar alunos passando os valores de
nome,
		 * matrícula, CRA e e-mail no ato da criação. Para isso,
precisamos
		 * definir um método construtor que me permita isso. Caso
façamos
		 * isso, o construtor "padrão" (o sem parâmetros que apenas
cria o
```

```
             * objeto da classe) deixa de existir. Para existir
novamente, preci-
             * samos redefiní-lo manualmente.
             *
             */

            // Aluno aluno = new Aluno() // Essa definição não existe
mais

            Aluno a1 = new Aluno("Matheus", "202830", 9f,
"matheus@email.com");
            Aluno a2 = new Aluno("João", "200000", 8f,
"joao.joao@email.com");
            Aluno a3 = new Aluno("Maria", "873932", 9.5f,
"maria@maria.com");

            /* Caso observarmos o método, veremos que ele calcula o
seguinte
            *
            *  return (this.cra + outro.cra)/2;
            *
            *  this.cra diz respeito ao cra do objeto mencionado antes
do
            *  ponto e outro.cra diz respeito ao cra do objeto passado
para
            *  o método
            *
            */

            // Nesse caso, this.cra diz respeito ao cra de a1
            a1.mediaCRAOutroAluno(a2);
            // Nesse caso, this.cra diz respeito ao cra de a2
            a2.mediaCRAOutroAluno(a1);
            a1.mediaCRAOutroAluno(a3);

            // Não podemos fazer isso, pois ClasseDeExemplo não consegue
            // acessar o nome de um aluno diretamente (no caso, consegue
            // indiretamente, através de getNome().

            //System.out.println(a1.nome);

            System.out.println(a1.getNome());

            // setCRA de aluno trata os casos em que o CRA passado é
menor
            // que 0 ou maior que 10, limitando esses valores. Dessa
maneira,
            // outras classes não conseguem setar valores incorretos ao
aluno

            a1.setCRA(18);
```

```
      }

}




/*
Aluno a = new Aluno("José", "20000000", 8.5f,
"jose.jose@aluno.uepb.edu.br");
//System.out.println(a.nome); Não consigo fazer acesso direto
//System.out.println(a.getNome());

a.setCRA(12);
System.out.println(a.getCRA());
*/



// Se "senha" for private, não consigo acessar os campos diretamente
aqui

//System.out.println(s.senha);
//s.senha = "123";
//System.out.println(s.senha);
```

## Código #04 - Aluno, mas com atributos e métodos estáticos

```
// Definindo a especificação de Aluno (atributos e métodos)

public class Aluno {

      // Definindo os atributos da classe Aluno
      // Nós não queremos que outras classes acessem ou manipulem esses
      // valores diretamente. Por conta disso, faremos com que apenas a
      // própria classe Aluno acesse e manipule diretamente esses
atributos.
      // Fazemos isso definindo-as como privadas (private)


      /* Aqui, definimos um atributo estático chamado inst
(instituição).
       * Ela fica em um local na memória, fora dos objetos da classe
       * Aluno em si. No caso, ela poderá ser acessada utilizando-se o
       * nome da classe seguido de ponto seguido de inst (Aluno.inst).
       */

      private static String inst = "UEPB";
```

```
      private String nome;
      private String matricula;
      private float cra;
      private String email;

      // Construtor da classe Aluno
      // Podemos definir os valores de nome, matricula, cra e email no
ato
      // da criação do objeto.
      // O método construtor tem o mesmo nome da classe e cria um
objeto da
      // classe Aluno.
      // Caso nenhum construtor tivesse sido definido, poderíamos
utilizar
      // um construtor padrão, sem parâmetros, que não define nenhum
valor
      // e apenas cria esse objeto dessa classe.


      public Aluno(String n, String m, float c, String e)
      {
            // o "this" diz respeito ao próprio objeto. Nesse caso,
this.nome
            // diz respeito ao nome do objeto que está sendo criado.

            this.nome = n;
            this.matricula = m;
            this.cra = c;
            this.email = e;
      }


      // Definindo os métodos da classe Aluno. Para chamar esse método,
      // precisamos de um objeto da classe Aluno. Sendo, por exemplo,
esse
      // objeto um aluno "a", faremos a.exibeInfo() para exibir as
infos de a

      public void exibeInfo()
      {
            System.out.printf("Nome: %s\nMatrícula: %s\nCRA:
%.2f\nE-mail: %s\n", this.nome, this.matricula, this.cra, this.email);
      }

      // Método público que me permite acessar o valor do nome do
objeto
      // Aluno em outra classe. Esse acesso é indireto, ou seja, outra
      // classe não consegue acessar o nome do aluno diretamente
(a.nome).
```

```
	// Ela só conseguirá fazer isso "consultando" o aluno, que cede
essa
	// informação através do getNome. É uma forma de conseguir
encapsulamento,
	// já que o atributo nome está encapsulado, protegido, não pode
ser
	// acessado por outras classes.

	public String getNome()
	{
		return this.nome;
	}

	// Método que permite modificarmos o nome do aluno. Novamente,
essa
	// modificação é indireta. Outra classe só consegue mudar o nome
do
	// aluno através desse método (ou outros métodos públicos que
permiti-
	// ssem essa mudança). Nesse caso, o valor é mudado independente
de
	// qual nome passarmos, mas poderíamos fazer algum tratamento.

	public void setNome(String n)
	{
		this.nome = n;

	}

	public float getCRA()
	{
		return cra;
	}

	// O método setCRA chama um método auxiliar que faz o tratamento
de
	// CRAs inválidos. Lembre que getCRA faz uma determinada função,
mas
	// outras classes não precisam ter detalhes disso
(encapsulamento,
	// ocultação de informação). Assim como utilizamos vários métodos
Java
	// que sabemos o que fazem mas não como fazem (e muitas vezes não
nos
	// importamos com isso).

	public void setCRA(float c)
	{
		cra = trataCRA(c);
	}
```

```
	// O método trataCRA é um método auxiliar que faz um
subprocedimento
	// dentro de aluno, e esse subprocedimento é irrelevante para
outras
	// classes (ou podemos forçar que não possam ser utilizados em
outras
	// classes). Conseguimos isso utilizando o modificador de acesso
	// private para o método.

	private float trataCRA(float c)
	{
		if(c >= 0 && c <= 10)
			return c;
		else if(c < 0)
			return 0;
		else
			return 10;
	}

	// Esse exemplo mostra a utilização do this. Podemos utilizar
this
	// para deixar de forma bem clara que o primeiro valor diz
respeito
	// ao objeto a partir do qual estamos invocando o método.

	public float mediaCRAOutroAluno(Aluno outro)
	{
		return (this.cra + outro.cra)/2;
	}

	// Lembrar de fazer get e set para os atributos da classe. Em
algumas
	// situações específicas, pode ser que vocês acabem criando
apenas
	// o get, apenas o set, ou até mesmo nenhum

	/* Aqui, temos os métodos get e set para obtermos a instituição.
	 * Poderiam não ser static também (mas aí precisaríamos criar um
	 * objeto Aluno para utilizá-las).
	 */

	public static String getInst()
	{
		return inst;
	}

	public static void setInst(String novoInst)
	{
		inst = novoInst;
	}
```

```
	/* O código abaixo está comentado pois não compila. Nós não
podemos
	 * utilizar "nome" (que é um atributo de instância, cada objeto
tem
	 * o seu) em um método estático (que não está ligado a objetos da
	 * classe). Experimente usar "this.nome" e verá que também não
	 * funciona.
	 */

	/*
	public static String getNomeStatic()
	{
		return nome;
	}
	*/
}

public class ExemploAulaStaticOverload {

	public static void main(String[] args)
	{
		SomaInteiros somador = new SomaInteiros();

		/* Esta é uma maneira de utilizarmos o método soma. Neste, o
		 * método soma foi definido como um método de instância
(não-
		 * estático), o que significa que precisamos de um objeto da
		 * classe SomaInteiros para utilizá-lo.
		 *
		 * Apesar de funcionar, neste caso, esse método parece pouco
		 * prático, já que precisamos criar um objeto de
SomaInteiros
		 * mas apenas para utilizar uma funcionalidade que nada
depende
		 * dos objetos em si.
		 *
		 */

		System.out.println(somador.soma(12, 8));

		/* Esta é outra maneira de utilizarmos o método soma (aqui,
chamado
		 * somaStatic). Neste caso, o método é estático, ou seja, é
		 * utilizado a partir da classe e não de instâncias da
classe.
		 * Aqui, não precisamos criar nenhum objeto do tipo
SomaInteiros
		 * para utilizar o método.
		 *
		 */
```

```
            System.out.println(SomaInteiros.somaStatic(12, 8));
      }

}
public class SomaInteiros {

      /* Aqui, temos a definição de um método chamado soma. Este
retorna
       * int e recebe dois parâmetros, ambos int.
       */

      public int soma(int a, int b)
      {
            System.out.println("O método soma com 2 param. foi
chamado");
            return a + b;
      }

      /* Aqui, temos a definição de outro método chamado soma. Este
retorna
       * int e recebe três parâmetros, todos int.
       *
       */

      public int soma(int a, int b, int c)
      {
            System.out.println("O método soma com 3 param. foi
chamado");
            return a + b + c;
      }

      /* Por mais que o tipo de retorno seja diferente, o nome e os
parâme-
       * tros deste método abaixo coincidem com o primeiro definido na
       * classe. Quando o método soma com dois inteiros é chamado, não
dá
       * para saber qual é o método que deve ser chamado: se é o que
retorna
       * int ou o que não retorna.
       *
       * No caso, não podemos definir o método abaixo e o primeiro
juntos.
       *
       */

       /*
       public void soma(int a, int b)
       {

       }
```

```
       */

      /* Abaixo, criamos um método soma que recebe dois inteiros e
retorna
       * um int, referente à soma deles. No caso, o método é estático.
Isso
       * significa que podemos utilizar o método a partir da classe, e
não
       * necessariamente de instâncias da classe.
       *
       * Nesse caso, esse tipo de implementação faz sentido, visto que
       * nós queremos apenas utilizar uma funcionalidade, sem
precisarmos,
       * por exemplo, alterar valores de um objeto, etc.
       *
       */

       public static int somaStatic(int a, int b)
       {
             return a + b;
       }
}
```

## Código #05 - Códigos envolvendo arrays

```
import java.util.ArrayList;

public class ExemploArraysListas {

      /* Podemos definir arrays passando o tamanho deles, mas podemos
       * também definir os seus elementos iniciais (e, por tabela, o
       * tamanho deles)
       */

      /* Ao definir um array passando o número de elementos, utilizamos
       * a palavra-reservada new, o tipo do array e o número de
elementos
       * entre colchetes.
       */

      /* As variáveis array1 e array2 são static pois queremos
       * utilizá-las sem a necessidade de criar um objeto do tipo da
       * classe.
       */

      static int[] array1 = new int[10];

      /* Uma outra forma de definir arrays é passando os elementos
iniciais.
```

```
       * No caso abaixo, o array terá tamanho 3, sendo o primeiro
elemento
       * o 10, o segundo o 8 e o terceiro o 6.
       */

      static int[] array2 = {10, 8, 6};

      /* Caso queiramos criar um array de strings, podemos fazer um
utilizando
       * o tipo String. No caso abaixo, cabem 5 strings no array.
       */

      static String[] nomes = new String[5];


      public static void main(String[] args)
      {
            /* Podemos fazer acesso indexado em um array, passando o
índice
             * que desejamos acessar. Lembrando que, assim como em C ou
             * Python, a indexação começa em 0.
             */

            System.out.println("O 2° elemento de array2 é: " +
array2[1]);

            /* Da mesma forma que podemos acessar, nós podemos alterar
os
             * valores dos arrays.
             */

            array2[1] = 12;

            System.out.println("O 2° elemento de array2 agora é: " +
array2[1]);

            /* Nós podemos obter o tamanho de um array utilizando
length. Tome
             * cuidado que, no caso, length não é um método, então não
utilizamos
             * parênteses.
             */

            System.out.println("O tamanho de array2 é: " +
array2.length);

            /* O length pode ser muito útil quando queremos, por
exemplo, iterar
             * pelos elementos de um array.
             */
```

```
            for(int i = 0; i < array2.length; i++)
            {
                  System.out.printf("O %d-ésimo elemento de array2 é
%d\n", i+1, array2[i]);
            }

            /* Podemos combinar os arrays com loops e estruturas de
controle
             * (e.g. if/else). No exemplo abaixo, veremos quais
elementos de
             * array2 são pares.
             */

            System.out.println();
            array2[1] = 7;
            for(int i = 0; i < array2.length; i++)
            {
                  if(array2[i] % 2 == 0)
                        System.out.printf("O %d-ésimo elemento de array2
é par\n", i+1);
                  else
                        System.out.printf("O %d-ésimo elemento de array2
é ímpar\n", i+1);
            }

            /* Quando lidamos com métodos, podemos passar parâmetros
para ele.
             * Esses parâmetros podem ser, por exemplo, variáveis de
tipos
             * primitivos, tipos criados por nós ou arrays. Abaixo,
definimos
             * um método que recebe um int (tipo primitivo) e soma 1 a
ele.
             * Vamos ver então se essa soma dentro do método mudou o
valor
             * fora dele.
             */

            int numero = 10;
            System.out.println("A variável numero armazena " + numero);
            somaUmEmInteiro(numero);
            System.out.println("A variável numero armazena " + numero);

            /* Ao executar os dois prints, veremos que numero não teve
valor
             * alterado. Isso acontece pois, como numero é um tipo
primitivo,
             * a passagem dele é feito por valor, isso é, cria-se uma
cópia
             * do valor de numero, e é essa cópia que teve valor
modificado.
```

```
            * O valor original continua o mesmo (mas o retorno do
método
            * ainda sim é 11, nesse caso).
            */

           /* Quando lidamos com arrays de inteiros, o que acontece é
            * que, de fato, os valores são alterados, visto que array
de
            * inteiros não é um tipo primitivo como int é.
            */

           /* OBS: não vimos esse tipo de situação nas aulas. Elas não
            * serão cobradas nessa prova, mas deixei aqui a título de
            * curiosidade e ressaltando que exploraremos mais isso mais
            * adiante.
            */

           System.out.println("Os valores de array2 antes de chamar o
método de soma");
           for(int i = 0; i < array2.length; i++)
           {
                 System.out.printf("O %d-ésimo elemento de array2 é
%d\n", i+1, array2[i]);
           }

           somaUmEmArray(array2);
           System.out.println("Os valores de array2 depois de chamar o
método de soma");
           for(int i = 0; i < array2.length; i++)
           {
                 System.out.printf("O %d-ésimo elemento de array2 é
%d\n", i+1, array2[i]);
           }

           /* Em Java, podemos criar listas com tamanhos dinâmicos, com
            * diversas operações diferentes, semelhantes às listas
Python.
            * Um tipo que permite isso em Java é o array list. Para
utilizar
            * o ArrayList, precisamos importar (import
java.util.ArrayList).
            *
            * Nós podemos definir o tipo de elementos que o array list
contém
            * passando esse tipo entre <>. Porém, não passamos tipos
            * primitivos para eles. Para termos, por exemplo, um array
list
            * de inteiros, precisamos passar sua classe "empacotadora",
no
            * inglês, wrapper.
            */
```

```
            ArrayList<Integer> listaNumeros = new ArrayList<Integer>();

            /* Abaixo, o uso de alguns métodos de array list*/

            // add(n): adiciona o elemento n informado ao array list
            listaNumeros.add(10);
            listaNumeros.add(14);
            listaNumeros.add(28);

            //add(int index, n): adiciona n na index-ésima posição da
lista
            listaNumeros.add(1, 12); //12 é adicionado na posição 1 (2°)

            //contains(n): true se o array list contém n
            System.out.println("listaNumeros contém 10? " +
listaNumeros.contains(10));

            //get(index): retorna o elemento na index-ésima posição
            System.out.println("Na posição 1 temos " +
listaNumeros.get(1));

            //indexOf(n): retorna o índice com a primeira ocorrência de
n
            // caso não tenha o elemento, retorna -1
            System.out.println("A 1ª ocorrência de 14 é :" +
listaNumeros.indexOf(14));

            //remove(index): retira da lista o elemento na index-ésima
posição
            listaNumeros.remove(2);

            //size(): retorna o número de elementos do array list
            System.out.println("O número de elementos de listaNumeros é
" + listaNumeros.size());


      }

      /* Método estático simples que recebe um número, soma 1 a ele e
       * o retorna. Nós veremos que...
       */

      public static int somaUmEmInteiro(int n)
      {
            n = n + 1;
            return n;

      }

      public static void somaUmEmArray(int[] numeros)
```

```
	{
		for(int i = 0; i < numeros.length; i++)
		{
			numeros[i] += 1;
		}
	}

}
```

## Código #06 - Hierarquia da classe Animal

```
/* Aqui, temos a definição da classe Animal. Essa classe será a super-
 * classe de outros animais, como Gato e Cachorro. Nessa classe, nós
 * definiremos os atributos e métodos comuns a todos os animais que
 * herdarem dessa classe. Dessa maneira, nós definimos esses atributos
 * e métodos uma vez (embora que, as vezes, nós podemos ou queremos
 * redefinir alguns métodos ou criar novos atributos).
 *
 * Quando falamos "animal", nós podemos "pensar" em muitos animais
 * diferentes. Quando falamos "gato", nós pensamos em um gato. Quando
 * falamos "cachorro", nós pensamos em um cachorro. Podemos fazer com
que
 * não seja permitido a criação de objetos do tipo Animal, o que pode
ser
 * justificado pela razão descrita acima.
 *
 * Nota importante: nós queremos que Animal seja criado e utilizado de
 * forma a comportar os atributos e métodos comuns aos animais, de modo
 * que possamos reutilizar em classes que herdem dela. Assim, a criação
 * da classe Animal faz sentido. O que não queremos é que Animal em si
 * seja instanciável, e podemos fazer isso utilizando a
palavra-reservada
 * abstract na definição da classe Animal.
 *
 * Muito do por que fazemos isso vem por conta do polimorfismo, mas
 * ainda veremos esse conteúdo, que acabou sendo movido para a Unidade
II
 * por conta do nosso calendário.
 *
 */


public abstract class Animal {

	/* O atributo nomeCientifico é protected, o que significa que
esse
	 * atributo pode ser acessado de forma direta por classes no
mesmo
	 * package dessa classe, além também de ser acessível diretamente
	 * pelas subclasses.
```

```
       *
       */

      protected String nomeCientifico;

      /* Na classe Animal, queremos definir o método falar. A questão é
que
       * não faz tanto sentido definir aqui uma implementação para o
método,
       * já que não sabemos qual o som que o Animal faz (já que o
próprio
       * conceito de "animal" é meio vago, não sabemos exatamente de
que
       * animal se trata). Caso definíssemos alguma implementação a
esse
       * método aqui, ele será substituído nas subclasses.
       *
       * Nós podemos fazer com que um método seja abstrato. Isso
significa
       * que esse método não tem implementação na classe. A
consequência é,
       * sendo definido como abstrato, faz-se com que as subclasses da
       * classe Animal sejam obrigadas a definir alguma implementação
para
       * esse método caso queiram ser instanciáveis (se não
implementarem,
       * essas subclasses também serão abstratas). No caso, como o
método
       * falar é abstrato, as classes que herdem de Animal (Gato e
Cachorro,
       * no caso) tenham que dar alguma implementação para ela.
       *
       * Nós poderíamos dar uma implementação ao método falar aqui, mas
       * no caso deixar esse método abstrato faz mais sentido, já que
       * não faz sentido uma implementação "padrão", além do que
queremos
       * "forçar a barra" para que as subclasses redefinam o que é o
falar
       * para ela (cachorros latem, gatos miam, outros fazem silêncio).
       *
       * Esse tipo de recurso também está bastante relacionado a
polimorfis-
       * mo, que veremos mais adiante.
       */

      public abstract void falar();

      /* Um detalhe interessante em Java (e outras linguagens OO) é que
todas
       * as classes herdam de Object, de maneira direta ou indireta.
Sendo
```

```
       * assim, essas classes herdam alguns métodos de Object, sendo um
       * deles o toString(). Nós podemos redefinir o método toString
nas
       * subclasses. Apague essa implementação abaixo e execute o
exemplo
       * de herança para ver o que sai no toString().
       *
       * Nós podemos utilizar @Override para indicar que esse método
está
       * sendo sobreposto (overridden) nessa classe. Sendo assim,
quando
       * chamamos toString(), não usamos mais a implementação de Object
e
       * sim de Animal.
       */

      @Override
      public String toString()
      {
            return "Eu sou um animal";
      }

}

/* A classe Cachorro herda da classe Animal. Nós fazemos isso
escrevendo
 * na definição da classe a palavra-reservada extends.
 *
 * Dessa maneira, a classe irá herdar os atributos e comportamentos
 * de Animal que estiverem marcados como public ou protected. Esses
 * atributos herdados não precisam ser redefinidos nessa classe.
 *
 */

public class Cachorro extends Animal {

      /* Cachorro tem o atributo nomeCientifico, mesmo que ele não
esteja
       * definido nessa classe. Isso acontece pois essa classe herda de
       * Animal, que tem esse atributo marcado como protected.
       */

      /* O método morder está sendo definido em Cachorro. Quando
dizemos que
       * a classe Cachorro estende (extends) Animal, uma interpretação
       * que podemos ter é a de que Cachorro estende o Animal, podendo
       * introduzir novos atributos e novos métodos (além dos herdados
da
       * classe Animal). A classe Animal não tem esse método definido.
       *
```

```
       */

      public void morder()
      {
            System.out.println("O cachorro mordeu");
      }

      /* Aqui, estamos dando uma implementação ao método falar, que é
       * abstrato na superclasse de Cachorro. Dessa maneira, a classe
poderá
       * ser instanciada, já que ela define uma implementação para um
método
       * abstrato herdado. Aqui, damos uma implementação que faz
sentido
       * para essa classe. Gato fará o mesmo.
       */

      @Override
      public void falar() {
            System.out.println("au au");

      }

}

/* A classe Gato herda da classe Animal. Nós fazemos isso escrevendo
 * na definição da classe a palavra-reservada extends.
 *
 * Dessa maneira, a classe Gato irá herdar os atributos e
comportamentos
 * de Animal que estiverem marcados como public ou protected. Esses
 * atributos herdados não precisam ser redefinidos nessa classe.
 *
 */

public class Gato extends Animal {

      /* Gato tem o atributo nomeCientifico, mesmo que ele não esteja
       * definido nessa classe. Isso acontece pois essa classe herda de
       * Animal, que tem esse atributo marcado como protected.
       */


      /* O método arranhar está sendo definido em Gato. Quando dizemos
que
       * a classe Gato estende (extends) a classe Animal, uma
interpretação
       * que podemos ter é a de que a classe Gato estende o Animal,
podendo
```

```
       * introduzir novos atributos e novos métodos (além dos herdados
da
       * classe Animal). A classe Animal não tem esse método definido.
       *
       */

      public void arranhar()
      {
            System.out.println("O gato arranhou");
      }

      /* Aqui, estamos dando uma implementação ao método falar, que é
       * abstrato na superclasse de Gato. Dessa maneira, essa classe
poderá
       * ser instanciada, já que ela define uma implementação para um
método
       * abstrato herdado. Aqui, damos uma implementação que faz
sentido
       * para essa classe. Cachorro fará o mesmo.
       */

      @Override
      public void falar() {
            System.out.println("miau");

      }

      @Override
      public String toString()
      {
            return "Eu sou um " + this.nomeCientifico;
      }





}
```

## Código #07 - Exemplos utilizando interfaces

```
package LinguagemDeProgramacao2;

/** A explicação mais longa para interfaces está em EmissorDeSom. Aqui,
 * temos um outro contrato, dessa vez, o de emitir imagem. O Pet não
irá
 * implementar essa interface, mas a TV sim.
 *
 */
```

```
public interface EmissorDeImagem {

      void emitirImagem();

}

package LinguagemDeProgramacao2;

/** Abaixo, nós temos uma interface. Uma interface estabelece um tipo
de
 * "contrato", que outras classes poderão obedecer.
 *
 * O contrato é estabelecido na forma dos métodos que a interface
contém.
 * Por padrão, esses métodos são públicos, então nem é necessário
definir
 * os métodos na interface como "public".
 *
 * Para uma classe obedecer ao contrato, precisa dar implementação a
todos
 * os métodos definidos na interface.
 *
 * O uso de interfaces permite, assim como herança, a utilização de
poli-
 * morfismo, fazendo com que uma chamada a método possa ter diferentes
 * resultados, a depender de a partir de quem você está chamando o
método.
 *
 * Uma vantagem muito interessante das interfaces em relação à herança
é
 * que uma classe Java pode implementar mais de uma interface, enquanto
ela
 * só pode herdar (diretamente) de uma classe.
 *
 * No exemplo abaixo, definimos a interface EmissorDeSom (nas aulas,
estava
 * utilizando o nome EmissorDeSom). Essa interface estabelece um
contrato que
 * diz que, para implementar essa interface, uma classe precisa dar uma
 * implementação a todos os métodos definidos dentro dela.
 *
 * No nosso caso, para implementar essa interface, a classe precisa ter
e
 * dar implementação ao método emitirSom().
 *
 * A parte interessante (que veremos em outros códigos nesse exemplo) é
que
 * EmissorDeSom pode ser implementado por classes que sejam muito
diferentes
```

```
 * entre si. Dessa maneira, podemos ter desde um Gato emitindo um som,
até
 * uma Pessoa, até um eletrodoméstico (e.g. um televisor).
 *
 * Essas coisas são bem diferentes, mas tem em comum a capacidade de
emitir
 * sons. Essas classes poderão ter atributos e métodos bem diferentes,
mas
 * implementarão o método emitirSom(), como parte do contrato que se
 * estabeleceu com EmissorDeSom.

*/


public interface EmissorDeSom {

      /** Lembrando: Não precisamos utilizar public na definição do
método
       * na interface - ele é público por padrão.
       *
       * Para implementar a interface, as classes que assim desejem
precisam
       * dar uma implementação ao método emitirSom().
       */

      void emitirSom();

}

package LinguagemDeProgramacao2;

public class Hamster extends Pet {

      /** O construtor de Hamster chama o construtor da superclasse,
para
       * atribuir valores aos atributos de nome e idade.
       */

      public Hamster(String nome, int idade) {
            super(nome, idade);
      }

      @Override
      public void emitirSom() {
            System.out.println("squeak");

      }

}
```

```
package LinguagemDeProgramacao2;

/** Aqui, temos a classe Pet, representando um animal doméstico. Essa
classe
 * é abstrata por motivos que já discutimos, tendo então um método
abstrato
 * de emitir som. Repare que foi possível definir o método emitirSom()
nessa
 * classe, mesmo que na interface não esteja nenhuma definição de
método
 * abstrato (o que não faria tanto sentido).
 *
 * Repare também que os atributos dessa classe são totalmente
diferentes
 * dos atributos de TV. Isso é possível e é um dos pontos positivos do
uso
 * de interfaces. Em comum com TV, eles atendem a um serviço: emitir
som.
 *
 * A classe Pet implementa EmissorDeSom. Veremos que não precisamos
fazer
 * com que as subclasses de Pet implementem a interface explicitamente,
na
 * definição dessas classes. Quando uma classe herda de outra que
implementa
 * uma interface, essa classe também implementa a interface.
 *
 */

public abstract class Pet implements EmissorDeSom{

      private String nome;
      private int idade;

      /** O método emitirSom() não tem sentido de ser implementado em
Pet, já
       * que constantemente iremos substituir essa implementação em
classes
       * derivadas (ou subclasses). Por conta disso, esse método foi
definido
       * como abstract e a responsabilidade de implementação dele foi
"jogado"
       * para as subclasses.
       *
       */

      public abstract void emitirSom();


      /** Os métodos abaixo não estão definidos na interface, mas
evidentemente
```

```
       * essa classe pode implementar esses métodos. Entretanto,
veremos
       * que não podemos invocar esses métodos em certas situações
(verão
       * no main).
       *
       * Os métodos abaixo serão reaproveitados em subclasses, então
podemos
       * já definir aqui em Pet.
       *
       */

      public Pet(String nome, int idade)
      {
            this.nome = nome;
            this.idade = idade;
      }

      public String getNome()
      {
            return this.nome;
      }

      public void setNome(String nome)
      {
            this.nome = nome;
      }

      public int getIdade()
      {
            return this.idade;
      }

      public void setIdade(int idade)
      {
            if(idade < 0)
            {
                  idade = 0;
            }

            this.idade = idade;
      }

}

package LinguagemDeProgramacao2;

/** A classe TV é uma classe que implementa a interface EmissorDeSom.
Isso
```

```
 * significa que ela pode fornecer a funcionalidade de emitir som. Se
não
 * definirmos esse método na TV, teremos um erro de compilação.
 *
 * Reparem que essa classe é bem diferente das outras que também
implementam
 * a interface, mas podemos utilizar elas dentro de certos contextos em
que
 * elas são unidas pelo serviço prestado: o de emitir sons.
 *
 * A classe TV também implementa a interface EmissorDeImagem. Nós
definimos
 * que interfaces uma classe implementa separando-as por vírgula.
 *
 */

public class TV implements EmissorDeSom, EmissorDeImagem {

      /** Aqui, estão os atributos da classe TV. A interface não
"força" a
       * definição de atributos em si (embora possamos utilizar métodos
para
       * sugerir isso), então reparem que os atributos de uma TV serão
       * completamente diferentes dos atributos de outras classes que
também
       * implementam a interface.
       */

      private String marca;
      private String modelo;

      /** Aqui, temos a implementação do método que consta na
interface. Agora,
       * o contrato está cumprido (essa classe deu uma implementação
para um
       * método da interface).
       */

      public void emitirSom()
      {
            System.out.printf("Emitindo som de TV de marca %s e modelo
%s", marca, modelo);
      }

      public void emitirImagem()
      {
            System.out.printf("Emitindo imagem de TV de marca %s e
modelo %s", marca, modelo);
      }
```

```
	/** Os métodos abaixo não estão definidos na interface, mas
evidentemente
	 * essa classe pode implementar esses métodos. Entretanto,
veremos
	 * que não podemos invocar esses métodos em certas situações
(verão
	 * no main).
	 *
	 */

	public TV(String marca, String modelo)
	{
		this.marca = marca;
		this.modelo = modelo;
	}

	public String getMarca()
	{
		return this.marca;
	}

	public void setMarca(String marca)
	{
		this.marca = marca;
	}

	public String getModelo()
	{
		return this.modelo;
	}

	public void setModelo(String modelo)
	{
		this.modelo = modelo;
	}



}

package LinguagemDeProgramacao2;

import java.util.ArrayList;
import java.util.List;

public class ProgramaTesteInterfaces {

	public static void main(String[] args)
	{
```

```
            /** Abaixo, temos a criação de uma lista para comportar
objetos, no
             * caso, que implementam a interface EmissorDeSom. Nós
podemos
             * adicionar a essa lista objetos de tipos bem diferentes
entre si
             * mas que atendem ao mesmo requisito: o de emitir som.
             *
             * emissoresDeSom foi definido como um List<EmissorDeSom>. A
própria
             * List é uma interface (nós veremos mais sobre isso mais
adiante)
             *
             */

            List<EmissorDeSom> emissoresDeSom = new
ArrayList<EmissorDeSom>();

            /** Abaixo, estamos adicionando Hamsters e TVs a essa lista.
             * Isso é possível, já que essas classes implementam a
interface.
             */

            Hamster h1 = new Hamster("Chiquinho", 2);
            TV tv1 = new TV("LF", "X-26A");

            emissoresDeSom.add(h1);
            emissoresDeSom.add(tv1);

            /** Abaixo, teremos uma chamada polimórfica de emitir som.
Caso
             * o objeto seja uma TV, será emitido som do jeito da TV.
Caso seja
             * um Hamster, será emitido som do jeito do Hamster.
             */

            for(EmissorDeSom emissor : emissoresDeSom)
            {
                  emissor.emitirSom();
            }

      }

}
```

**Código #08 - Comparando herança, interface e composição**

```
package LinguagemDeProgramacao2;
```

```
/** Abaixo, temos a definição de um Animal. Como discutimos em sala,
apesar
 * de podermos fazer essa classe dessa maneira e, nas subclasses, fazer
o
 * override dos métodos, não há grandes benefícios em fazer essa classe
 * abstrata.
 *
 * O principal motivo disso é que todos os métodos da classe são
abstratos,
 * então nenhum código é reaproveitado nas classes derivadas
(subclasses).
 * O reaproveitamento de código é um dos principais motivos para nós
 * utilizarmos herança.
 *
 * Lembre que uma classe Java só herda diretamente de uma classe Java.
Se
 * fizermos da maneira abaixo, caso uma classe herde de Animal, não
poderá
 * herdar de mais ninguém, e isso pode ser indesejado.
 *
 */

public abstract class Animal {

	public abstract void falar();
	public abstract void mover();
	public abstract void reproduzir();

}

package LinguagemDeProgramacao2;

/** Abaixo, temos a representação de um Coelho, que herda de Animal. Já
 * que um Coelho não se torna outro Animal, podemos fazer uso de
herança de
 * maneira mais confiável e adequada.
 */

public class Coelho extends Animal {

	@Override
	public void falar() {
		// TODO implementação de fala de coelho
	}

	@Override
	public void mover() {
		// TODO implementação de movimentação de coelho
	}
```

```
	@Override
	public void reproduzir() {
		// TODO implementação de reprodução de coelho

	}

}

package LinguagemDeProgramacao2;

/** Abaixo, temos a representação de um Rato, que herda de Animal. Já
 * que um Rato não se torna outro Animal, podemos fazer uso de herança
de
 * maneira mais confiável e adequada.
 */

public class Rato extends Animal{

	@Override
	public void falar() {
		// TODO implementação da fala de um rato

	}

	@Override
	public void mover() {
		// TODO Auto-generated method stub

	}

	@Override
	public void reproduzir() {
		// TODO Auto-generated method stub

	}

}

package LinguagemDeProgramacao2;

import java.util.ArrayList;
import java.util.List;

/** Abaixo, temos uma classe que representa uma Pessoa. No nosso
exemplo,
 * fizemos uma pessoa que possui os atributos de nome e de lista de
funções.
 *
 * No caso abaixo, utilizamos o recurso de composição, que indica que
uma
```

```
 * classe possui um atributo que remete a outra classe. A composição
estabe-
 * lece uma relação "tem um", de modo que uma Pessoa tem uma lista de
Funcao.
 *
 * Em Funcao, você encontrará mais explicações sobre como ela funciona.
 *
 * Caso tivéssemos criado Pessoa como uma classe base para Professor e
para
 * Coordenador, quando criássemos um professor, não poderíamos fazer
com
 * que ele se tornasse coordenador, já que coordenador estaria do outro
lado
 * da hierarquia. Apesar de podermos utilizar upcast e downcast, não
daria
 * certo fazer isso para "transformar" um professor em coordenador,
assim
 * as pessoas não poderiam mudar seus papeis.
 *
 *                          Pessoa
 *                       |    |
 *               Professor    Coordenador
 *
 *
 * Para resolver isso, desassociamos o papel que a pessoa exerce, de
modo
 * que, ao invés de um coordenador ser uma pessoa (Coordenador extends
 * Pessoa), fizemos com que a Pessoa possa "ter uma" (composição)
função,
 * sendo uma dessas funções a de coordenador. A função em si é
representada
 * por uma interface, que estabelece que as classes que implementam ela
 * forneçam uma maneira de calcular o salário e de obter o salário
base.
 *
 */

public class Pessoa {

      /** Os atributos de Pessoa no exemplo. Em situações reais, é
bastante
       * provável que mais atributos (e métodos get e set) seriam
definidos,
       * mas vamos manter esses dois para fins de exemplo. Poderíamos
também
       * utilizar apenas um atributo Funcao ao invés de uma lista.
       */

      private String nome;
      private List<Funcao> funcoes;
```

```
	/** Aqui, definimos o método de calcular o salário de uma pessoa. No
	 * nosso exemplo, o salário da pessoa é calculado como função do salário
	 * dessa pessoa nas funções que ela exerce. Cada função terá um cálculo
	 * de salário diferente, o que funciona abaixo por conta do polimorfismo.
	 */

	public float calculaSalario()
	{
		float total = 0;
		for(Funcao funcao : funcoes)
		{
			total += funcao.calculaSalario();
		}

		return total;
	}

	public Pessoa(String nome)
	{
		this.nome = nome;
		this.funcoes = new ArrayList<Funcao>();
	}

	public String getNome()
	{
		return this.nome;
	}

	public void setNome(String novoNome)
	{
		this.nome = novoNome;
	}

	public List<Funcao> getFuncao()
	{
		return this.funcoes;
	}

	public void setFuncao(List<Funcao> novasFuncoes)
	{
		this.funcoes = novasFuncoes;
	}

	public void atribuirFuncao(Funcao funcao)
	{
		this.funcoes.add(funcao);
	}
```

```
}

package LinguagemDeProgramacao2;

public interface Funcao {

	float calculaSalario();

	/** No material que postei em interfaces, comentei que interfaces
não
	 * contém atributos nem forçam atributos. O caso abaixo não força
o
	 * atributo salarioBase, mas sugere que a classe tenha alguma
coisa que
	 * represente isso, e esse valor deve ser retornado. No nosso
caso,
	 * o set de salário base não está sendo forçado.
	 */

	float getSalarioBase();


}

package LinguagemDeProgramacao2;

/** Aqui, temos uma classe que representa um coordenador de computação.
No
 * nosso exemplo, uma Pessoa pode ser professora e coordenadora.
 *
 */

public class CoordenadorDeComputacao implements Funcao{

	private static float salarioBaseCoordenador = 300;
	private int cargaHoraria;
	private float bonus;

	public float calculaSalario() {
		return salarioBaseCoordenador * cargaHoraria * bonus;
	}

	public CoordenadorDeComputacao(int cargaHoraria, float bonus)
	{
		this.cargaHoraria = cargaHoraria;
		this.bonus = bonus;
	}

	public float getSalarioBase()
```

```
	{
		return salarioBaseCoordenador;
	}

	public static void setSalarioBase(float novoSalarioBase)
	{
		salarioBaseCoordenador = novoSalarioBase;
	}

	public int getCargaHoraria()
	{
		return this.cargaHoraria;
	}

	public void setCargaHoraria(int novaCargaHoraria)
	{
		this.cargaHoraria = novaCargaHoraria;
	}

	public float getBonus()
	{
		return this.bonus;
	}

	public void setBonus(float novoBonus)
	{
		bonus = novoBonus;
	}

}

package LinguagemDeProgramacao2;

/** A classe abaixo representa um professor de Computação, em uma
universi-
 * dade. Ela possui alguns atributos e alguns métodos que serão
explicados
 * mais adiante. Assim como vimos na parte de interfaces, podemos fazer
com
 * que classes que implementem a mesma interface sejam bem diferentes
entre
 * si, mas em comum elas tem algum serviço em comum (no caso, fornecer
um
 * modo de calcular o salário).
 *
 */

public class ProfessorDeComputacao implements Funcao {
```

```
	/** Abaixo, temos o atributo salarioBaseProfessor. Ele foi
definido como
	 * estático pois, caso ele mude, essa mudança se refletirá em
todos os
	 * professores de Computação. Caso tivéssemos esse atributo como
sendo
	 * não-estático, então cada objeto professor de computação teria
um
	 * valor próprio para esse atributo, o que poderia dificultar a
mudança
	 * em todos, caso acontecesse.
	 *
	 */

	private static float salarioBaseProfessor = 100;

	/** A carga horária de professores pode variar entre si, então
esse
	 * atributo ficou sendo não-estático mesmo.
	 *
	 */
	private int cargaHoraria;

	public float calculaSalario() {
		return salarioBaseProfessor * this.cargaHoraria;
	}

	public ProfessorDeComputacao(int cargaHoraria)
	{
		this.cargaHoraria = cargaHoraria;
	}

	public float getSalarioBase()
	{
		return salarioBaseProfessor;
	}

	public static void setSalarioBase(float novoSalarioBase)
	{
		salarioBaseProfessor = novoSalarioBase;
	}

	public int getCargaHoraria()
	{
		return this.cargaHoraria;
	}

	public void setCargaHoraria(int novaCargaHoraria)
	{
		this.cargaHoraria = novaCargaHoraria;
	}
```

```
}

package LinguagemDeProgramacao2;

/** No código abaixo, iremos fazer com que uma pessoa possa receber
funções,
 * perder funções e fazer com que o código funcione normalmente.
 *
 */

public class ProgramaHerIntComp {

      public static void main(String[] args)
      {

            // Criando uma Pessoa chamada Maria
            Pessoa maria = new Pessoa("Maria");

            // Dando a essa pessoa a função de professora de computação,
com
            // 20 horas semanais.
            maria.atribuirFuncao(new ProfessorDeComputacao(20));

            // Calculando quanto Maria ganha, com uma função
            System.out.printf("Maria ganha R$%.2f de salário\n",
maria.calculaSalario());

            // Dando outra função para Maria - a de coordenadora de
computação,
            // com 20 horas semanais, sem bônus (1, se for 0 zera o
salário dela)
            maria.atribuirFuncao(new CoordenadorDeComputacao(20, 1));

            // Calculando quanto Maria ganha, agora com duas funções.
            System.out.printf("Maria ganha R$%.2f de salário\n",
maria.calculaSalario());
      }

}
```

## Código #09 - Exemplos com mapas

```
package LinguagemDeProgramacao2;

/* Nessa seção do código, realizamos os imports necessários. Não
precisam
 * lembrar de fazer esses imports na prova. Assuma que, caso sejam
 * necessários em algum enunciado, eles já estarão importados.
 *
```

```
 */

import java.util.Map;
import java.util.HashMap;
import java.util.Scanner;
import java.util.Set;
import java.util.HashSet;
import java.util.List;
import java.util.ArrayList;

public class ExemploMapa {

      public static void main(String[] args)
      {
            Scanner scan = new Scanner(System.in);

            /* No trecho abaixo, criamos um mapa (Map é uma interface)
             * que armazena chaves do tipo String e valores também do
             * tipo String (por isso <String, String>).
             */

            Map<String, String> mapaMatriculas;

            /* Nós inicializamos mapaMatriculas utilizando um HashMap,
que
             * é uma classe que implementa a interface de Map.
             *
             */

            mapaMatriculas = new HashMap<String, String>();

            /* O método put adiciona um par chave-valor ao mapa. No
             * caso, o primeiro parâmetro é uma String, assim como o
             * segundo parâmetro também.
             *
             */

            mapaMatriculas.put("12345", "Fulano de Souza Alves");
            mapaMatriculas.put("23456", "Sicrano Oliveira Menezes");
            mapaMatriculas.put("34567", "Beltrano Silva Morais");

            /* O método replace troca o valor do par cuja chave é
"34567"
             * para "Beltrano de Silva Morais".
             *
             */

            mapaMatriculas.replace("34567", "Beltrano Morais", "Beltrano
de Silva Morais");
```

```
            /* O método size retorna o número de pares chave-valor no
mapa
             *
             */

            System.out.println(mapaMatriculas.size());

            System.out.println("Digite uma matrícula: ");
            String entrada = scan.next();

            /* O método containsKey recebe um valor e retorna true caso
             * aquele valor seja chave de algum par chave-valor do mapa.
             * Caso contrário, retorna false.
             */

            if(mapaMatriculas.containsKey(entrada))
            {

                  /* O método get de mapa recebe um parâmetro e retorna
o
                   * valor cuja chave é o que for passado. Caso o que
for
                   * passado não seja chave de algum par chave-valor, o
método
                   * retorna "null" (por isso é importante a verificação
                   * com containsKey).
                   */

                  System.out.println(mapaMatriculas.get(entrada));
            }
            else
            {
                  System.out.println("Não há aluno com a matrícula
informada");
            }


            /* Aqui, criamos um Set para armazenar apenas as chaves do
             * mapa criado anteriormente. Através desse Set, faremos
             * as iterações pelas chaves, para assim obter os valores.
             *
             * Lembre-se que, nos Sets, não existem duplicatas de
elementos
             * nem uma ordem entre eles.
             */

            Set<String> matriculas = mapaMatriculas.keySet();

            /* É uma forma de iterar sobre elementos de Sets. Aqui, não
             * podemos utilizar acesso pelo índice, já que os elementos
```

```
             * não tem ordem, então podemos utilizar esse tipo de
iteração
             * (o chamado "foreach").
             *
             */

            for(String matricula : matriculas)
            {
                  System.out.println(mapaMatriculas.get(matricula));
            }





      }

}
```

**Código #10 - Exemplos envolvendo testes**

```
package teste;

public class Aluno {

      private String nome;
      private float nota1;
      private float nota2;
      private float percFaltas;

      public Aluno(String nome)
      {
            if(nome == "")
                  this.nome = "Default";
            else
                  this.nome = nome;
      }

      public Aluno(String nome, float nota1, float nota2)
      {
            this.nome = nome;
            this.nota1 = nota1;
            this.nota2 = nota2;
      }

      public String getNome()
      {
            return this.nome;
      }
```

```
    public float getNota1()
    {
        return this.nota1;
    }

    public float getNota2()
    {
        return this.nota2;
    }

    public float percFaltas()
    {
        return this.percFaltas;
    }

    public float media()
    {
        return (this.nota1 + this.nota2)/2;
    }

    public boolean estaReprovadoPorFalta()
    {
        return this.percFaltas > 0.25;
    }

    public void setFaltas(float percFaltas)
    {
        this.percFaltas = percFaltas;
    }

}
```